13.1 会话模型设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250,000 tokens，输出 ~18,000 tokens（本章合计）

会话（Session）是 OpenClaw 中对话上下文的容器。每次和 AI 的交互都发生在某个会话里，会话保存了对话历史、配置覆盖、元数据等信息。本节分析会话模型的核心设计。

13.1.1 会话键（Session Key）的结构

OpenClaw 里每个会话由一个**会话键**（Session Key）唯一标识，标准格式是：

```
agent:<agentId>:<rest>
```

其中：

- `agent` 是固定前缀，标识这是一个 Agent 会话键
- `<agentId>` 是 Agent 的标识符（如 `main` 、 `coding-assistant` ）
- `<rest>` 是会话的具体标识部分，根据消息类型和配置不同而变化

`src/sessions/session-key-utils.ts` 中的 `parseAgentSessionKey()` 函数负责解析这个结构：

```typescript
// src/sessions/session-key-utils.ts
export type ParsedAgentSessionKey = {
  agentId: string;
  rest: string;
};

export function parseAgentSessionKey(
  sessionKey: string | undefined | null,
): ParsedAgentSessionKey | null {
  const raw = (sessionKey ?? "").trim();
  if (!raw) return null;
  const parts = raw.split(":").filter(Boolean);
  if (parts.length < 3) return null;       // 至少需要 3 段
  if (parts[0] !== "agent") return null;   // 必须以 "agent" 开头
  const agentId = parts[1]?.trim();
  const rest = parts.slice(2).join(":");   // 剩余部分可能包含冒号
  if (!agentId || !rest) return null;
  return { agentId, rest };
}
```

几种典型的会话键：

会话键	含义
`agent:main:main`	默认 Agent 的主会话
`agent:main:dm:alice`	默认 Agent 与 alice 的 DM 会话
`agent:main:telegram:group:12345`	默认 Agent 在 Telegram 群组 12345 的会话
`agent:main:slack:dm:U123:thread:T456`	Slack DM 中某个线程的会话
`agent:coding:subagent:task-1`	coding Agent 的子 Agent 会话
`agent:main:cron:daily-report:run:uuid`	定时任务的运行会话

Agent ID 规范化

Agent ID 在存储前会经过严格的规范化处理：

```
// src/routing/session-key.ts
const VALID_ID_RE = /^[a-z0-9][a-z0-9_-]{0,63}$/i;
const INVALID_CHARS_RE = /[^a-z0-9_-]+/g;

export function normalizeAgentId(value: string | undefined | null): string {
  const trimmed = (value ?? "").trim();
  if (!trimmed) return DEFAULT_AGENT_ID; // "main"
  if (VALID_ID_RE.test(trimmed)) return trimmed.toLowerCase();
  // 最大努力回退：将非法字符替换为 "-"
  return trimmed.toLowerCase()
    .replace(INVALID_CHARS_RE, "-")
    .replace(/^-+/, "")  // 去除前导 "-"
    .replace(/-+$/, "")  // 去除尾部 "-"
    .slice(0, 64) || DEFAULT_AGENT_ID;
}
```

这个设计保证了会话键是**路径安全**（Path-Safe）和**Shell 安全**（Shell-Friendly）的——因为会话键最终会被用作文件路径的一部分。

特殊会话键

除了标准的 `agent:*:*` 格式，还有两个特殊的会话键：

- `global` — 全局会话，所有 Agent 共享
- `unknown` — 未识别来源的消息会话

辅助函数可以检测特殊类型的会话键：

```
// 检测是否为子 Agent 会话
export function isSubagentSessionKey(sessionKey: string): boolean {
  const raw = (sessionKey ?? "").trim();
  if (raw.toLowerCase().startsWith("subagent:")) return true;
  const parsed = parseAgentSessionKey(raw);
  return Boolean((parsed?.rest ?? "").toLowerCase().startsWith("subagent:"));
}

// 检测线程会话并提取父会话键
export function resolveThreadParentSessionKey(sessionKey: string): string | null {
  const normalized = sessionKey.toLowerCase();
  const markers = [":thread:", ":topic:"];
  let idx = -1;
  for (const marker of markers) {
    const candidate = normalized.lastIndexOf(marker);
    if (candidate > idx) idx = candidate;
  }
  if (idx <= 0) return null;
  return sessionKey.slice(0, idx).trim() || null;
}
```

13.1.2 会话范围（DM Scope）

用户通过私聊（DM, Direct Message）和 OpenClaw 交互时，有个关键问题：**同一个用户在不同通道（Telegram、Slack、Discord 等）上的对话，应该共享同一个会话，还是各自独立？**

OpenClaw 通过 `dmScope` 配置项提供了四种策略：

范围	会话键格式	行为
`main`	`agent:<id>:main`	所有 DM 共享同一个会话
`per-peer`	`agent:<id>:dm:<peerId>`	每个发送者一个会话
`per-channel-peer`	`agent:<id>:<channel>:dm:<peerId>`	每个通道 +发送者一个会话
`per-account-channel-peer`	`agent:<id>:<channel>:<account>:dm:<peerId>`	最细粒度

`buildAgentPeerSessionKey()` 函数根据 `dmScope` 生成对应的会话键：

```typescript
// src/routing/session-key.ts
export function buildAgentPeerSessionKey(params: {
  agentId: string;
  mainKey?: string;
  channel: string;
  accountId?: string | null;
  peerKind?: "dm" | "group" | "channel" | null;
  peerId?: string | null;
  dmScope?: "main" | "per-peer" | "per-channel-peer" | "per-account-channel-peer";
  identityLinks?: Record<string, string[]>;
}): string {
  const peerKind = params.peerKind ?? "dm";

  if (peerKind === "dm") {
    const dmScope = params.dmScope ?? "main";
    let peerId = (params.peerId ?? "").trim().toLowerCase();

    // 身份链接：将多个平台的用户 ID 映射到一个规范 ID
    const linkedPeerId = resolveLinkedPeerId({ ... });
    if (linkedPeerId) peerId = linkedPeerId;

    if (dmScope === "per-account-channel-peer" && peerId) {
      return `agent:${agentId}:${channel}:${accountId}:dm:${peerId}`;
    }
    if (dmScope === "per-channel-peer" && peerId) {
      return `agent:${agentId}:${channel}:dm:${peerId}`;
    }
    if (dmScope === "per-peer" && peerId) {
      return `agent:${agentId}:dm:${peerId}`;
    }
    // dmScope === "main"
    return buildAgentMainSessionKey({ agentId, mainKey });
  }

  // 群组/频道消息
  return `agent:${agentId}:${channel}:${peerKind}:${peerId}`;
}
```

> **衍生解释**：身份链接（Identity Links）是一种跨平台用户身份合并机制。在 `per-peer` 或更细粒度的范围下，同一个人在 Telegram（ID: `tg:12345` ）和 Slack（ID: `slack:U678` ）上会被视为不同的发送者，产生不同的会话。通过配置 `identityLinks` ，可以将这些 ID 映射到同一个规范身份，使得该用户在所有通道上共享同一个会话上下文。

13.1.3 会话持久化：JSON Store + JSONL 转录文件

OpenClaw 使用**双层存储**模型来持久化会话：

第一层：会话元数据（JSON Store）

会话的元数据（标签、配置覆盖、时间戳等）存储在一个 JSON 文件中，路径通常为 `<stateDir>/agents/<agentId>/sessions.json` 。每个会话键对应一个 `SessionEntry` 对象：

```
// src/gateway/session-utils.types.ts（简化展示）
export type GatewaySessionRow = {
  key: string;                      // 会话键
  kind: "direct" | "group" | "global" | "unknown";
  label?: string;                   // 用户可见的标签
  displayName?: string;             // 显示名
  channel?: string;                 // 所属通道
  updatedAt: number | null;         // 最后更新时间
  sessionId?: string;               // UUID 格式的会话 ID
  thinkingLevel?: string;           // 思考级别覆盖
  model?: string;                   // 模型覆盖
  sendPolicy?: "allow" | "deny";    // 发送策略
  // ... 更多字段
};
```

第二层：对话转录（JSONL Transcript）

实际的对话历史（每条消息的内容、角色、工具调用结果等）存储为 JSONL（JSON Lines）格式的文件。每行是一个 JSON 对象，代表一条消息。JSONL 格式的优势在于：

1. **追加友好**：新消息只需追加一行，不需要读取和重写整个文件
2. **流式读取**：可以逐行读取，不需要将整个文件加载到内存
3. **部分读取**：可以只读取前 N 行或后 N 行

转录文件的路径与会话键相关，存储在 Agent 的工作目录下。 `session-utils.fs.ts` 提供了读取转录文件的工具函数：

```
// session-utils.fs.ts 导出的函数（从 session-utils.ts 重新导出）
export {
  readFirstUserMessageFromTranscript,   // 读取第一条用户消息（用于派生标题）
  readLastMessagePreviewFromTranscript,  // 读取最后一条消息预览
  readSessionPreviewItemsFromTranscript, // 读取预览条目
  readSessionMessages,                   // 读取完整消息列表
  resolveSessionTranscriptCandidates,    // 解析转录文件路径候选
};
```

会话分类

`classifySessionKey()` 函数根据会话键的结构判断会话类型：

```
export function classifySessionKey(
  key: string, entry?: SessionEntry
): "direct" | "group" | "global" | "unknown" {
  if (key === "global") return "global";
  if (key === "unknown") return "unknown";
  if (entry?.chatType === "group" || entry?.chatType === "channel") return "group";
  if (key.includes(":group:") || key.includes(":channel:")) return "group";
  return "direct";
}
```

本节小结

OpenClaw 的会话模型设计有几个核心特点：

1. **结构化的会话键**：`agent:<agentId>:<rest>` 格式将 Agent 身份、消息来源、通道信息编码进键名，使得会话键本身就是可解析的元数据
2. **灵活的 DM 范围**：四种范围策略覆盖了从"所有人共享"到"每个通道每个账户每个用户独立"的完整频谱
3. **双层持久化**：JSON Store 存元数据（快速查询），JSONL 存对话历史（高效追加），各取所长
4. **身份链接**：跨通道的用户身份合并，避免同一个人在不同平台产生碎片化的会话

下一节我们看会话路由的源码，看看 Gateway 如何根据入站消息的来源自动决定使用哪个会话。